

Bloque 04_linux

Índice de módulos

- Linux capabilities
- Tareas cron escribibles
- Docker y escapes de contenedor
- Enumeracion para privesc en Linux
- Grupos peligrosos (docker, lxd, disk)
- Exploits de kernel (ultimo recurso)
- NFS con no_root_squash
- Secuestro de PATH
- Python library hijacking
- Abuso de sudo
- Sudo con env_keep (LD_PRELOAD)
- Binarios SUID
- Inyeccion por comodin (wildcard)
- Ficheros sensibles escribibles

Linux capabilities

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Las capabilities parten los privilegios de root en piezas concretas que se asignan a un binario sin darle SUID completo. Es una version mas granular del suid: en vez de 'corre como root', es 'puede hacer esta cosa de root'. Algunas capabilities permiten escalar directamente.

Se usan para que un binario haga una tarea privilegiada (abrir un puerto bajo, cambiar de uid) sin ser root del todo. Pero cap_setuid permite a un interprete (python, perl) cambiar su uid a 0, y cap_dac_read_search permite leer cualquier fichero. Mal puestas, son root.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeracion (enum-privesc-linux), justo despues de SUID. getcap recorre el sistema buscandolas.

Cómo identificarla

- getcap lista un binario con cap_setuid+ep (escalada directa via python/perl).
- cap_dac_read_search+ep: lectura de /etc/shadow y cualquier fichero protegido.
- cap_setuid sobre un interprete no estandar puesto a proposito en el reto.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

getcap -r / 2>/dev/null y busca cap_setuid o cap_dac_read_search en la salida.

Evidencia esperada: Una linea 'binario = cap_setuid+ep' sobre un interprete (python/perl); tras el abuso, euid=0.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
getcap -r / 2>/dev/null
```

Objetivo y contexto: Lista todos los binarios con capabilities asignadas. -r recorre recursivamente; el ruido de permiso denegado se oculta con 2>/dev/null.

Resultado esperado: Líneas 'binario = cap_xxx+ep'. cap_setuid -> escalada directa (GTF0Bins).
cap_dac_read_search -> lees /etc/shadow. Ignora los del sistema base.

Capa 2: prueba documentada

```
/usr/bin/python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

Objetivo y contexto: Si python tiene cap_setuid, setuid(0) cambia tu uid a root y luego lanzas una shell que ya corre como root. Es la receta de GTF0Bins para esa capability.

Resultado esperado: Prompt de root y id con uid=0. Cambia python por el interprete concreto que getcap marco con cap_setuid.

Anatomía de los payloads

La primera prueba es `getcap -r / 2>/dev/null`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una línea 'binario = cap_setuid+ep' sobre un interprete (python/perl); tras el abuso, `euid=0`.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Recorre el sistema con `getcap -r` buscando capabilities en binarios.
- Identifica la capability y el binario en GTF0Bins (sección Capabilities).
- Si es `cap_setuid` en python/perl, fuerza `setuid(0)` y lanza una shell.
- Confirma `uid=0` y recoge la flag.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `getcap -r /` no devuelve ningún binario fuera de los estándar del sistema.
- Las capabilities presentes no incluyen `cap_setuid` ni `cap_dac_read_search` (son inofensivas para `privesc`).

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Permisos finos que dan a un binario poderes de root concretos; cap_setuid sobre un interprete es escalada directa. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(capabilities\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Tareas cron escribibles

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

cron ejecuta tareas programadas, a menudo como root. Si una de esas tareas llama a un script, binario o ruta que tu usuario puede escribir (o si usa comodines o rutas relativas manipulables), puedes inyectar tu propio comando y esperar a que cron lo ejecute con privilegios. Es el privesc-modelo aplicado al tiempo.

Los admins programan mantenimiento (backups, limpiezas) como root y a veces dejan el script en una ruta escribible por otros, o usan comodines (tar *) que un atacante puede secuestrar creando ficheros con nombres especiales. cron lo ejecuta tal cual, sin validar quien toco el script.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando sudo (sudo-abuse) y SUID (suid) no dieron nada. cron es mas lento porque hay que esperar a que la tarea corra, pero es un vector muy comun en rooms.

Cómo identificarla

- Un script referenciado en cron es escribible por tu usuario o su grupo.
- La tarea usa una ruta relativa o un comodin (*) en un directorio donde puedes crear ficheros.
- Aparecen procesos que arrancan cada pocos minutos (pspy lo revela) corriendo como root.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Comprueba con ls -l si el script que ejecuta root es escribible por tu usuario o grupo.

Evidencia esperada: Permiso de escritura (w) para ti en el script; tras inyectar tu payload y esperar el ciclo de cron, una shell/binario root.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
cat /etc/crontab; ls -la /etc/cron.*
```

Objetivo y contexto: Muestra las tareas programadas del sistema y con que frecuencia corren. Es el primer sitio donde mirar que ejecuta root de forma automática.

Resultado esperado: Líneas con horario, usuario (root) y el comando/script. Anota los scripts que corren como root: el siguiente paso es ver si puedes escribirlos.

Capa 2: prueba documentada

```
ls -l /ruta/al/script_de_cron.sh
```

Objetivo y contexto: Comprueba si el script que ejecuta root es escribible por ti. Ese permiso de escritura es justo la frontera de confianza rota que necesitas.

Resultado esperado: Los permisos y el dueño. Si tu usuario o un grupo tuyo tiene w, puedes inyectar comandos. Si no, busca comodines o rutas relativas manipulables.

Anatomía de los payloads

La primera prueba es `cat /etc/crontab; ls -la /etc/cron.*`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Permiso de escritura (w) para ti en el script; tras inyectar tu payload y esperar el ciclo de cron, una shell/binario root.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Lee las tareas programadas del sistema y de otros usuarios.
- Comprueba permisos de los scripts/binarios que invocan (`ls -l`): busca los escribibles.
- Inyecta tu payload (una reverse shell o el bit SUID a bash) en el punto controlable.
- Espera a que cron ejecute y recoge tu shell/binario root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ninguna tarea root referencia un script o ruta que puedas escribir.

- Los scripts invocados usan rutas absolutas y permisos correctos (root:root, 700) sin comodines.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si root ejecuta periodicamente un script o binario que tu puedes modificar, tu código correrá como root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- Concepto fuente de THM Fieldbook (cron-abuse)
- Referencia técnica externa

Navegación

Anterior: Construcción de payloads. Índice: curso. Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Docker y escapes de contenedor

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un contenedor comparte kernel con el host. El acceso al socket Docker, un contenedor privilegiado o un bind mount del filesystem del host puede convertir una shell en el contenedor en control del host.

El daemon Docker actúa con privilegios del host y los montajes trasladan esos permisos al proceso dentro del contenedor.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Tras obtener shell en un contenedor o cuando id/groups muestra docker y el socket /var/run/docker.sock es accesible.

Cómo identificarla

- docker.sock
- --privileged
- CAP_SYS_ADMIN
- mount /host
- grupo docker

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Identifica socket, capacidades, mounts y privilegios; confirma impacto solo con un fichero de prueba no destructivo.

Evidencia esperada: Sabes si el contenedor es una frontera real o una vía de escritura/lectura sobre el host.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
id; ls -l /var/run/docker.sock; capsh --print 2>/dev/null
```

Objetivo y contexto: Recoge grupo, socket y capacidades antes de intentar cualquier acción.

Resultado esperado: Permisos del socket y capabilities efectivas.

Capa 2: prueba documentada

```
docker ps --format '{{.ID}} {{.Image}} {{.Mounts}}'
```

Objetivo y contexto: Comprueba si puedes hablar con el daemon y que montajes existen.

Resultado esperado: Contenedores y montajes visibles al usuario actual.

Anatomía de los payloads

La primera prueba es `id; ls -l /var/run/docker.sock; capsh --print 2>/dev/null`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Sabes si el contenedor es una frontera real o una vía de escritura/lectura sobre el host.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica runtime y usuario.
- Revisa socket, capabilities y mounts.
- Prueba acceso a un recurso de laboratorio.
- Registra la remediación: retirar socket, privilegios y mounts innecesarios.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El proceso está en una VM sin Docker accesible.
- El socket no existe y no hay privilegios especiales ni montajes del host.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Distinguir aislamiento normal de montajes, sockets y privilegios que permiten alcanzar el host. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (docker-container-escape)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion para privesc en Linux

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Enumerar para privesc es recorrer de forma sistemática todos los vectores posibles en un orden que va de lo más rentable a lo más laborioso. Herramientas como linpeas lo automatizan y colorean los hallazgos, pero entender que representa cada resultado es lo que convierte 'output' en 'vector'. Es el paso previo obligado del privesc-modelo.

Adivinar pierde tiempo. Un recorrido fijo (sudo, SUID, capabilities, cron, servicios, credenciales, kernel) garantiza que no te saltas nada. linpeas resalta en rojo/amarillo lo probable, pero da falsos positivos: hay que validar cada pista contra el modelo mental.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Justo tras conseguir shell y estabilizarla (tty-stabilization). Corre primero los checks rápidos manuales (sudo -l, id) y luego lanza linpeas para lo exhaustivo.

Cómo identificarla

- linpeas marca en rojo/amarillo (99% PE vectors) un binario, servicio o fichero.
- pspy muestra procesos o cron que arrancan como root de forma periódica.
- Un hallazgo que encaja con el modelo: algo root que puedes leer/escribir/influir.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Corre sudo -l e id primero (gratis); después linpeas para el resto.

Evidencia esperada: Al menos un hallazgo resaltado en rojo/amarillo en linpeas, o una entrada NOPASSWD en sudo -l.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
./linpeas.sh -a 2>/dev/null | tee linpeas.txt
```

Objetivo y contexto: Automatiza cientos de checks de privesc en un pase. -a es el modo agresivo; guardar la salida (tee) te deja releerla sin volver a correrlo.

Resultado esperado: Fijate en lo marcado en rojo/amarillo, sobre todo 'sudo', 'SUID', 'Capabilities', 'Cron', 'Interesting files'. Valida cada uno; hay falsos positivos.

Capa 2: prueba documentada

```
./pspy64 -pf -i 1000
```

Objetivo y contexto: Muestra procesos y comandos que arrancan sin necesitar root, ideal para ver cron y tareas que ps puntual no capta. Revela que ejecuta root de forma periodica.

Resultado esperado: Comandos que aparecen cada X segundos con uid=0. Si alguno llama a un script o binario que puedes escribir, tienes un vector tipo cron-abuse.

Anatomía de los payloads

La primera prueba es `./linpeas.sh -a 2>/dev/null | tee linpeas.txt`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Al menos un hallazgo resaltado en rojo/amarillo en linpeas, o una entrada NOPASSWD en sudo -l.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Checks rapidos manuales: id, sudo -l, SUID (suid), capabilities (capabilities).
- Lanza linpeas y lee de arriba a abajo, priorizando lo resaltado.
- Usa pspy para cazar tareas periodicas y procesos que no ves con ps puntual.
- Cada pista -> validala contra un vector concreto (cron, PATH, servicio) antes de explotar.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ya recorriste todos los vectores manualmente y linpeas no aporta nada nuevo.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Antes de explotar hay que enumerar en orden fijo; un script automatiza, pero saber que mira cada check es lo que te hace resolver. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (enum-privesc-linux)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Grupos peligrosos (docker, lxd, disk)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Algunos grupos conceden capacidades que equivalen a root aunque tu usuario no lo sea. docker y lxd permiten arrancar un contenedor que monta el sistema de ficheros del host y escribir en el como root. disk da acceso crudo al dispositivo (leer /etc/shadow con debugfs). adm permite leer todos los logs. Es el atajo mas rapido del privesc-modelo.

Estos grupos existen para administrar subsistemas, pero su poder se traduce trivialmente en root: si controlas el demonio de contenedores puedes montar / del host; si lees el disco crudo, saltas todos los permisos de fichero. Por eso id/groups es lo primero que se mira.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Nada mas orientarte con id/groups (privesc-modelo). Si sales en docker/lxd/disk/adm, prioriza este vector sobre todo lo demas.

Cómo identificarla

- id muestra tu usuario en docker, lxd/lxc, disk o adm.
- El binario docker/lxc responde (el demonio esta corriendo).
- Para disk: debugfs puede abrir el dispositivo raiz.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

id para ver tus grupos; si sale docker, prueba docker run -v /:/mnt --rm -it alpine chroot /mnt sh.

Evidencia esperada: Una shell dentro del chroot con acceso de escritura a / del host real.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

Objetivo y contexto: Si estas en el grupo docker, arrancas un contenedor que monta TODO el disco del host en /mnt y haces chroot: ya eres root sobre los ficheros reales del host.

Resultado esperado: Una shell root dentro del chroot con acceso a / del host. Desde ahí lees flags, editas /etc/passwd o pones SUID a bash en el host real.

Capa 2: prueba documentada

```
debugfs -w /dev/sda1
```

Objetivo y contexto: Con el grupo disk accedes al dispositivo en crudo, saltando permisos de fichero. debugfs -w permite leer y escribir el sistema de ficheros directamente.

Resultado esperado: Un prompt debugfs. 'cat /etc/shadow' desde ahí ignora permisos. Ajusta /dev/sdaX al dispositivo raíz real (mira lsblk/mount).

Anatomía de los payloads

La primera prueba es `docker run -v /:/mnt --rm -it alpine chroot /mnt sh`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una shell dentro del chroot con acceso de escritura a / del host real.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Mira tus grupos con id.
- docker: arranca un contenedor montando / del host y chroot para escribir como root.
- lxd: importa una imagen y montala con security.privileged para acceder al host.
- disk: usa debugfs sobre el dispositivo raíz para leer /etc/shadow o escribir.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- id no muestra ninguno de esos grupos.

- El demonio correspondiente (docker/lxd) no esta corriendo pese a estar en el grupo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Pertenecer a ciertos grupos es root casi directo: docker/lxd montan el disco del host; disk lo lee crudo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(group-abuse-linux\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Exploits de kernel (ultimo recurso)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un exploit de kernel aprovecha un fallo en el propio nucleo de Linux para ejecutar codigo como root desde un usuario normal. Casos famosos: DirtyCow, PwnKit (polkit), DirtyPipe. Funcionan cuando la version del kernel o de un componente del sistema es vulnerable, independientemente de la config.

Se dejan para el FINAL por dos razones: los exploits de kernel pueden causar un kernel panic y tumbar la maquina (arruinando la room), y muchas veces hay un vector de config mas limpio y estable. Solo cuando la enumeracion (enum-privesc-linux) no dio nada mejor, se recurre a esto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ningun vector de config (sudo, SUID, cron, capabilities) funciono y uname -r revela un kernel viejo o hay un componente vulnerable (polkit, sudo Baron Samedit).

Cómo identificarla

- uname -r muestra un kernel antiguo con CVE conocido.
- linpeas sugiere exploits de kernel aplicables.
- Componentes vulnerables presentes (pkexec para PwnKit, sudo < 1.9.5p2 para Baron Samedit).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara uname -a con CVEs conocidos (DirtyCow, PwnKit, DirtyPipe) para ese rango de version.

Evidencia esperada: Tras ejecutar el exploit compilado, un prompt root y uid=0. Si no aplica, la maquina puede quedar inestable: usalo con cuidado.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
uname -a; cat /etc/os-release
```

Objetivo y contexto: La version exacta del kernel y de la distro es lo que decide que exploit aplica. Un exploit para otra version puede no funcionar o tumbar la maquina.

Resultado esperado: Version del kernel (ej. 4.4.0) y distro. Cruzalo con searchsploit/CVEs; kernels < ~4.8 huelen a DirtyCow, cualquiera con pkexec vulnerable a PwnKit.

Capa 2: prueba documentada

```
searchsploit linux kernel <version>
```

Objetivo y contexto: Busca exploits locales publicos para esa version concreta de kernel. Prioriza los locales (Local Privilege Escalation) sobre los remotos.

Resultado esperado: Titulos con su rango de version. Lee el PoC antes de lanzarlo (exploitar-cve); comprueba que la version encaja para no arriesgar un panic.

Anatomía de los payloads

La primera prueba es `uname -a; cat /etc/os-release`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tras ejecutar el exploit compilado, un prompt root y uid=0. Si no aplica, la maquina puede quedar inestable: usalo con cuidado.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Saca la version exacta con `uname -a` y las versiones de sudo/polkit.
- Busca un exploit para esa version y leelo antes de ejecutarlo (exploitar-cve).
- Compila en una maquina compatible (o en la victima si hay gcc) y transfierelo.
- Ejecuta con cuidado; confirma uid=0. Si falla o cuelga, vuelve a los vectores de config.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El kernel/distro esta parcheado contra los CVEs conocidos para esa version.

- Quedan vectores de configuración sin probar (sudo, SUID, cron): prueba esos primero.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si el kernel es viejo, un exploit local eleva a root; deja esto para el final porque puede tumbar la máquina. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(kernel-exploits-linux\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

NFS con no_root_squash

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

no_root_squash es una opción de NFS que hace que el root de tu máquina sea tratado como root en el servidor NFS (en lugar de degradarlo a nobody). Si un share exportado con esa opción está montado en una ruta accesible en la víctima, montas el share desde tu Kali (donde SI eres root), creas ahí un binario SUID root, y lo ejecutas en la víctima para escalar.

Normalmente NFS 'aplasta' (squash) el root remoto para que no pueda actuar como root local. no_root_squash desactiva esa protección: los ficheros que creas como root en tu máquina conservan uid 0 en el servidor. Un binario SUID creado así corre como root al ejecutarlo en la víctima.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ves NFS (2049) o /etc/exports revela un share con no_root_squash. showmount lista los exports disponibles.

Cómo identificarla

- showmount -e muestra un export disponible.
- /etc/exports (legible en la víctima) contiene 'no_root_squash'.
- El export está montado o es montable en una ruta útil de la víctima.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Monta el share en tu Kali, crea un binario SUID como root, y ejecútalo desde la víctima por esa misma ruta.

Evidencia esperada: El binario mantiene el bit SUID y euid root al ejecutarse en la víctima.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
showmount -e $IP
```

Objetivo y contexto: Lista los directorios que el servidor NFS exporta y a quien. Es el primer paso: necesitas saber que share montar y si aplica.

Resultado esperado: Rutas exportadas y sus restricciones de host. Un export sin restricción (*) es montable por ti. Luego confirma `no_root_squash` en `/etc/exports`.

Capa 2: prueba documentada

```
sudo mount -t nfs $IP:/export /mnt/nfs && cd /mnt/nfs
```

Objetivo y contexto: Monta el share en tu Kali, donde eres root. Los ficheros SUID que crees aquí como root conservarán uid 0 en la víctima gracias a `no_root_squash`.

Resultado esperado: El share montado en `/mnt/nfs`. Ahora compila un binario que lance bash, hazlo `chown root:root` y `chmod +s`; luego ejecútalo en la víctima desde esa misma ruta.

Anatomía de los payloads

La primera prueba es `showmount -e $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El binario mantiene el bit SUID y euid root al ejecutarse en la víctima.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera los exports con `showmount -e` contra la IP.
- Monta el share en tu Kali (como root).
- Crea ahí un binario que lance una shell y ponle el bit SUID (como root de tu Kali).
- En la víctima, ejecuta ese binario desde la ruta del share: corre como root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `showmount -e` no devuelve ningún export, o el export tiene `root_squash` activo.
- No puedes montar NFS desde tu máquina de ataque (firewall, sin cliente NFS).
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un export NFS con no_root_squash te deja crear ficheros SUID como root desde tu maquina y ejecutarlos en la victima. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (nfs-no-root-squash)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Secuestro de PATH

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un programa privilegiado invoca otro comando por su nombre (por ejemplo 'cat' en vez de '/bin/cat'), el sistema lo busca por el PATH. Si puedes modificar el PATH o escribir en un directorio que aparezca antes, colocas ahí un binario malicioso con ese nombre y el proceso root ejecuta el tuyo. Otro caso del privesc-modelo.

El programador uso una ruta relativa por comodidad, confiando en que el PATH apunta a los binarios legítimos. Pero si controlas el PATH (o hay un directorio escribible en el), rompes esa confianza: tu binario se ejecuta con los privilegios del proceso.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un SUID (suid) o un script de cron-abuse que corre como root llama a comandos sin ruta absoluta. strings sobre el binario o leer el script lo revela.

Cómo identificarla

- strings de un SUID muestra un comando llamado sin ruta (system("service ...")).
- Un script root usa comandos sin path absoluto.
- Tienes un directorio escribible que puedes anteponer al PATH.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

strings sobre el SUID y busca llamadas a comandos sin '/' delante.

Evidencia esperada: Un nombre de comando sin ruta absoluta en las strings; tras crear tu binario falso y anteponerlo al PATH, ejecución como el proceso privilegiado.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
strings /ruta/al/suid
```

Objetivo y contexto: Revela cadenas del binario, incluidos comandos que llama. Un comando sin '/' es candidato a PATH hijacking; con ruta absoluta, no.

Resultado esperado: Nombres de comandos invocados. Si ves 'service', 'cat', 'ps' sin ruta, ese es tu punto: crea un binario con ese nombre en un dir que controles.

Capa 2: prueba documentada

```
cd /tmp && echo '/bin/bash' > cat && chmod +x cat && export PATH=/tmp:$PATH
```

Objetivo y contexto: Crea un 'cat' falso que lanza bash, y lo pone primero en el PATH. Cuando el proceso root llame a 'cat', ejecutara el tuyo con privilegios de root.

Resultado esperado: Al ejecutar despues el binario/script privilegiado, obtienes shell root. Restaura el PATH luego para no romper tu propia sesion.

Anatomía de los payloads

La primera prueba es `strings /ruta/al/suid`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un nombre de comando sin ruta absoluta en las strings; tras crear tu binario falso y anteponerlo al PATH, ejecucion como el proceso privilegiado.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Detecta el comando invocado sin ruta absoluta (strings o leyendo el script).
- Crea un binario/script con ese nombre que lance una shell (o ponga SUID a bash).
- Antepone tu directorio al PATH (export PATH=/tmp:\$PATH) y dale permiso de ejecucion.
- Ejecuta el binario/script privilegiado; correra tu version. Confirma uid=0.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El binario/script usa siempre rutas absolutas (/bin/cat, /usr/bin/service).

- El proceso privilegiado fija su propio PATH explícitamente antes de ejecutar (no hereda el tuyo).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si un script o SUID que corre como root llama a un binario sin ruta absoluta, colocas tu propio binario antes en el PATH. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (path-hijacking)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Python library hijacking

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un script de Python que se ejecuta como root hace 'import modulo', Python busca ese modulo recorriendo una lista de directorios (sys.path) en orden. Si puedes colocar un fichero .py con ese nombre en un directorio que se busque ANTES que el modulo real, o sobrescribir el propio modulo, tu codigo se ejecuta con privilegios de root al importarse. Es el path-hijacking aplicado al sistema de imports de Python.

sys.path incluye, y normalmente en primer lugar, el directorio del propio script; ademas de PYTHONPATH y los site-packages. El programador confia en que se importara el modulo legitimo, pero Python simplemente coge el primero que encuentra. Si el directorio del script es escribible, o el modulo importado lo es, o sudo conserva PYTHONPATH, rompes esa confianza. Misma logica que el privesc-modelo.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando puedes ejecutar un script Python como root (via sudo-abuse) o un cron root lo ejecuta, Y puedes escribir uno de los sitios donde Python busca sus modulos. Es un vector muy comun en rooms de escalada Linux.

Cómo identificarla

- sudo -l te deja ejecutar un script .py como root.
- El directorio del script (o un modulo que importa) es escribible por tu usuario.
- El script hace 'import' de un modulo con nombre poco comun que vive junto a el.
- sudo -l muestra env_keep+=PYTHONPATH.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Revisa sys.path y los permisos de cada directorio anterior al modulo real que importa el script.

Evidencia esperada: Un directorio escribible por ti apareciendo ANTES del modulo legitimo en sys.path.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
sudo -l
```

Objetivo y contexto: Punto de partida: ver si puedes ejecutar un script .py como root. Una entrada NOPASSWD sobre un python es el candidato típico de este vector.

Resultado esperado: Líneas '(root) NOPASSWD: /usr/bin/python3 /ruta/script.py'. Abre y lee ese script: te dice que módulos importa y donde vive.

Capa 2: prueba documentada

```
python3 -c 'import sys; print(sys.path)'
```

Objetivo y contexto: Muestra el orden en que Python busca módulos. El primer directorio escribible que preceda al módulo real es tu hueco para inyectar.

Resultado esperado: Lista de directorios. Si el dir del script (o " = dir actual) va primero y es escribible, tienes hijacking directo. Cruza esto con los permisos.

Capa 3: prueba documentada

```
echo 'import os; os.setuid(0); os.system("/bin/bash")' > /dir_escribible/modulo.py
```

Objetivo y contexto: Crea un módulo malicioso con el MISMO nombre que el que importa el script. Al ejecutarse el script como root, importa el tuyo y ejecuta tu código con uid 0.

Resultado esperado: Al lanzar 'sudo python3 /ruta/script.py', obtienes una shell root (id = uid=0). Ajusta el nombre del fichero al módulo exacto que importa el script.

Anatomía de los payloads

La primera prueba es `sudo -l`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un directorio escribible por ti apareciendo ANTES del módulo legítimo en `sys.path`.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Lee el script root e identifica que modulos importa.
- Mira sys.path y los permisos: ¿puedes escribir el dir del script, el modulo, o un dir que preceda al real?
- Crea un fichero con el nombre del modulo importado que ejecute tu payload (os.setuid(0) + shell).
- Ejecuta el script como root; al importar tu modulo, corre como root. Confirma con id (uid=0).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El script root no hace ningun 'import' de un modulo no estandar.
- Todos los directorios de sys.path anteriores al modulo real son de solo-lectura para ti.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.

3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un script Python que corre como root importa un modulo; si puedes escribir ese modulo (o uno que Python busque antes), tu código corre como root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- Concepto fuente de THM Fieldbook (python-library-hijacking)
- Referencia técnica externa

Navegación

Anterior: Construcción de payloads. Índice: curso. Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Abuso de sudo

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

sudo permite a un usuario ejecutar comandos concretos como otro (normalmente root). Si tu usuario tiene permitido ejecutar un binario que además puede lanzar una shell, leer/escribir ficheros o ejecutar código, ese permiso puntual se convierte en root total. Otra cara del privesc-modelo.

El admin concede sudo sobre un binario pensando que solo hace su función, pero muchos binarios tienen funciones secundarias (un editor puede abrir una shell, less puede ejecutar comandos). GTFOBins lista para cada uno como escapar. Además fallos como `env_keep` o rutas relativas amplían el abuso.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Es lo PRIMERO que se mira en privesc Linux por su alta rentabilidad. En cuanto tengas una shell estable (`tty-stabilization`), lanza `sudo -l`.

Cómo identificarla

- `sudo -l` muestra entradas '(root) NOPASSWD: /ruta/bin' (ni siquiera piden password).
- El binario permitido aparece en GTFOBins bajo 'sudo'.
- Ves `env_keep+=LD_PRELOAD` o rutas relativas: vectores extra de abuso.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Ejecuta `sudo -l` y busca cada binario listado en GTFOBins bajo 'sudo'.

Evidencia esperada: Líneas '(root) NOPASSWD: /ruta/binario'. Tras aplicar la receta de GTFOBins, un prompt # con `uid=0`.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`sudo -l`

Objetivo y contexto: Enumera exactamente que comandos puedes correr como otro usuario. Es el vector mas rapido y comun; muchas rooms se resuelven aqui sin buscar mas.

Resultado esperado: Entradas '(root) NOPASSWD: ...'. Cada binario permitido -> GTFOBins. Si pide password y no la tienes, este vector queda en pausa.

Capa 2: prueba documentada

`sudo /usr/bin/find . -exec /bin/sh \; -quit`

Objetivo y contexto: Ejemplo tipico de GTFOBins: si puedes sudo find, su flag -exec lanza una shell heredando el privilegio root de sudo.

Resultado esperado: Prompt # y id con uid=0. Cambia find por el binario concreto que te permita tu sudo -l; la idea (forzar una shell) es la misma.

Anatomía de los payloads

La primera prueba es `sudo -l`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Lineas '(root) NOPASSWD: /ruta/binario'. Tras aplicar la receta de GTFOBins, un prompt # con uid=0.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Ejecuta `sudo -l` para ver que tienes permitido y si es sin password.
- Busca el binario permitido en GTFOBins, seccion 'sudo'.
- Aplica la linea de escape (suele forzar una shell o ejecutar un comando como root).
- Confirma con id; si eres root, recoge la flag y anota el vector.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `sudo -l` pide password y no la tienes.
- El binario permitido no aparece en GTFOBins y no tiene funcion de escape conocida (editor sin !shell, etc).

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

sudo -l te dice que puedes ejecutar como root; muchos binarios permiten saltar de ahí a una shell root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (sudo-abuse)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Sudo con env_keep (LD_PRELOAD)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

LD_PRELOAD fuerza al cargador a inyectar una librería tuya antes que las del sistema en cualquier programa. Si sudo está configurado con env_keep+=LD_PRELOAD, esa variable sobrevive al sudo y puedes precargar una .so maliciosa en un comando que tengas permitido ejecutar con sudo, corriendo tu código como root. Variante avanzada de sudo-abuse.

El admin conserva variables de entorno por comodidad sin darse cuenta de que LD_PRELOAD permite inyectar código. Tu .so define una función (por ejemplo el constructor) que lanza una shell; al cargarla en el proceso privilegiado, esa shell es root.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando sudo -l (sudo-abuse) muestra 'env_keep+=LD_PRELOAD' (o LD_LIBRARY_PATH) y tienes algún comando permitido con sudo, aunque sea inofensivo.

Cómo identificarla

- sudo -l muestra 'env_keep+=LD_PRELOAD' en la política.
- Tienes al menos un comando ejecutable con sudo.
- Puedes compilar en la máquina (gcc presente) o subir la .so ya compilada.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Ejecuta el comando permitido con sudo LD_PRELOAD=/tu.so y observa si tu código corre.

Evidencia esperada: Un prompt de root al ejecutar el comando con la variable precargada.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
gcc -fPIC -shared -o /tmp/pe.so pe.c -nostartfiles
```

Objetivo y contexto: Compila tu librería como shared object. -shared la hace precargable; -nostartfiles evita dependencias que romperían la carga en el proceso víctima.

Resultado esperado: El fichero /tmp/pe.so. Su código (una función _init/constructor que hace setuid(0); system("/bin/bash")) se ejecutará al precargarse.

Capa 2: prueba documentada

```
sudo LD_PRELOAD=/tmp/pe.so <comando_permitido>
```

Objetivo y contexto: Al ejecutar el comando permitido con sudo, LD_PRELOAD (conservado por env_keep) carga tu .so en el proceso root y dispara tu shell.

Resultado esperado: Un prompt de root (id = uid=0). Si da error de carga, revisa que compilaste con -nostartfiles y que env_keep realmente conserva la variable.

Anatomía de los payloads

La primera prueba es gcc -fPIC -shared -o /tmp/pe.so pe.c -nostartfiles. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un prompt de root al ejecutar el comando con la variable precargada.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma env_keep+=LD_PRELOAD en sudo -l.
- Escribe una .so mínima cuyo constructor lance /bin/bash y quite privilegios (setuid(0)).
- Compíllala como shared object.
- Ejecuta un comando permitido con sudo LD_PRELOAD=/ruta/tu.so; obtienes shell root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- sudo -l no muestra env_keep para LD_PRELOAD/LD_LIBRARY_PATH.

- No tienes gcc en la maquina ni forma de subir una .so ya compilada.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si sudo conserva LD_PRELOAD, cargas una librería tuya en un comando permitido y ejecutas código como root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(sudo-ld-preload\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Binarios SUID

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

El bit SUID hace que un ejecutable corra siempre como su propietario, sin importar quien lo lance. Si el dueño es root y el binario permite ejecutar comandos, leer/escribir ficheros o lanzar otra shell, puedes abusar de el para actuar como root. Es una aplicación directa del *privesc*-modelo.

SUID existe para casos legítimos (*passwd* necesita tocar */etc/shadow*). El problema es cuando un binario con SUID tiene una función que permite salir a una shell o leer ficheros arbitrarios: entonces esa capacidad se ejecuta como root. GTFOBins cataloga exactamente que binarios son abusables y como.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la fase de *privesc* Linux, justo después de mirar *sudo*-abuse. Busca binarios SUID que NO sean del sistema base o cuya versión sea conocida por ser abusable.

Cómo identificarla

- find de SUID devuelve binarios raros (*nmap*, *find*, *cp*, *python*, *vim*, *tar*) fuera de los típicos.
- Un binario a medida del reto con el bit SUID puesto.
- El binario aparece en GTFOBins con la etiqueta SUID.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Busca el binario candidato en GTFOBins (sección SUID) y ejecuta su línea de abuso exacta.

Evidencia esperada: Un prompt nuevo con # en vez de \$, y *euid=0* en *id*.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
find / -perm -4000 -type f 2>/dev/null
```

Objetivo y contexto: Lista todos los binarios con bit SUID. -perm -4000 filtra justo ese bit; 2>/dev/null oculta el ruido de permiso denegado.

Resultado esperado: Rutas de binarios SUID. Ignora los típicos del sistema; cualquier cosa rara o a medida -> GTF0Bins. Anota también la versión si es un binario conocido.

Anatomía de los payloads

La primera prueba es `find / -perm -4000 -type f 2>/dev/null`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un prompt nuevo con # en vez de \$, y `euid=0` en id.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Lista los binarios SUID del sistema.
- Descarta los normales (`passwd`, `sudo`, `mount`, `ping`) y quedate con lo inusual.
- Busca cada candidato en GTF0Bins filtrando por 'SUID': te da la línea exacta.
- Ejecuta el abuso y confirma con id que tienes `euid=0`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Todos los SUID listados son binarios estándar del sistema sin función de escape conocida en GTF0Bins.
- El binario SUID es de tu propio usuario (dueño = tu uid, no root): no gana privilegio.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un binario SUID se ejecuta con los privilegios de su dueño (a menudo root), no con los tuyos. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (suid)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Inyeccion por comodin (wildcard)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Cuando un script root ejecuta algo como 'tar czf backup.tar.gz *' en un directorio, el shell expande el * a la lista de ficheros ANTES de pasarla al comando. Si puedes crear ficheros ahí, creas ficheros cuyo nombre es en realidad una opción del comando (como --checkpoint-action de tar), y el comando ejecuta lo que pongas.

El shell no distingue un fichero llamado '--checkpoint-action=exec=sh script.sh' de una opción legítima: el glob lo expande igual. tar, chown, rsync y otros tienen flags que ejecutan comandos, y así el proceso root corre tu código. Es un privesc-modelo via nombres de fichero.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando cron-abuse o un script root usa un comando con * en un directorio escribible por ti. Muy común con tar en tareas de backup.

Cómo identificarla

- Un cron/script root ejecuta tar/chown/rsync con * en una ruta que puedes escribir.
- El directorio del comodín es de tu propiedad o escribible por tu grupo.
- El comando tiene flags conocidas de ejecución (tar --checkpoint-action).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Crea los ficheros con nombres de flags (--checkpoint=1, --checkpoint-action=...) y espera al siguiente ciclo.

Evidencia esperada: Tras el ciclo, tu payload se ejecuta como root (comprueba con el binario/flag SUID que hayas preparado).

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > shell.sh
```

Objetivo y contexto: Prepara el payload que el comando privilegiado acabara ejecutando: copia bash con bit SUID para tener luego una shell root.

Resultado esperado: El fichero shell.sh en el directorio del comodin. Aun no hace nada: falta que el comando root lo invoque via las flags.

Capa 2: prueba documentada

```
touch './--checkpoint=1'; touch './--checkpoint-action=exec=sh shell.sh'
```

Objetivo y contexto: Crea ficheros cuyos nombres son opciones de tar. Cuando el cron ejecute 'tar ... *', el glob los mete como argumentos y tar ejecutara shell.sh como root.

Resultado esperado: Tras correr la tarea, /tmp/rootbash existe con bit SUID: lanzalo con -p para shell root. Ajusta las flags al comando concreto (chown/rsync tienen las suyas).

Anatomía de los payloads

La primera prueba es `echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' > shell.sh`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tras el ciclo, tu payload se ejecuta como root (comprueba con el binario/flag SUID que hayas preparado).

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica el comando con comodin y el directorio (leyendo el cron/script).
- Crea el payload que ejecutara (p. ej. un .sh que ponga SUID a bash).
- Crea ficheros cuyos NOMBRES sean las flags que hacen que el comando ejecute tu payload.
- Espera a que la tarea corra y recoge tu shell/binario root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El comando no usa comodines o los ficheros se listan explícitamente por nombre.

- El directorio del comodin no es escribible por tu usuario ni grupo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un comando root con * en un directorio donde puedes crear ficheros: nombras ficheros como flags y el shell los expande a argumentos. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(wildcard-injection\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Ficheros sensibles escribibles

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Ciertos ficheros del sistema definen quien es root y quien puede escalar. Si por una mala configuración de permisos tu usuario puede escribir en `/etc/passwd`, `/etc/shadow` o `/etc/sudoers`, puedes añadir o alterar una cuenta con uid 0 o darte permisos sudo. No hay exploit: es escritura directa a la fuente de la verdad.

Estos ficheros deberían ser solo-lectura para usuarios normales. Cuando alguien afloja los permisos (o un script los deja escribibles), rompes la frontera de confianza mas básica del sistema: `/etc/passwd` todavía permite un hash de password en el segundo campo, así que puedes crear un root con password conocida.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la enumeración (`enum-privesc-linux`) marca `/etc/passwd`, `/etc/shadow` o `/etc/sudoers` como escribibles por ti (`ls -l` o el check de linpeas).

Cómo identificarla

- `ls -l /etc/passwd` muestra permiso de escritura para tu usuario o grupo.
- `/etc/shadow` legible/escribible (linpeas lo resalta).
- Un fichero en `/etc/sudoers.d/` escribible por ti.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

`ls -l /etc/passwd /etc/shadow /etc/sudoers` y busca un permiso `w` para ti.

Evidencia esperada: Un `'w'` en los permisos de alguno de esos ficheros para tu usuario o grupo.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
ls -l /etc/passwd /etc/shadow /etc/sudoers
```

Objetivo y contexto: Comprueba de un vistazo si tienes escritura en los ficheros que definen identidad y privilegios. Es un check barato con recompensa enorme.

Resultado esperado: Los permisos. Una 'w' para tu usuario/grupo en cualquiera de ellos es escalada directa.

Normalmente solo root escribe; si no, hay mala config.

Capa 2: prueba documentada

```
openssl passwd -1 -salt xyz Passw0rd
```

Objetivo y contexto: Genera un hash valido para /etc/passwd. Si passwd es escribible, anades una linea 'hacker:HASH:0:0:/root:/bin/bash' y creas un root con password conocida.

Resultado esperado: El hash a pegar en el segundo campo. Tras anadir la linea, 'su hacker' con Passw0rd te da uid=0. -1 usa MD5crypt, compatible con /etc/passwd.

Anatomía de los payloads

La primera prueba es `ls -l /etc/passwd /etc/shadow /etc/sudoers`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un 'w' en los permisos de alguno de esos ficheros para tu usuario o grupo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Comprueba permisos de /etc/passwd, /etc/shadow y /etc/sudoers(.d).
- Si /etc/passwd es escribible, genera un hash (openssl passwd) y anade una linea de usuario con uid 0.
- Cambia a ese usuario con su y la password que pusiste.
- Confirma uid=0. Alternativa: darte una linea NOPASSWD en sudoers.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- /etc/passwd, /etc/shadow y /etc/sudoers(.d) son todos solo-lectura para tu usuario/grupo.
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si puedes escribir `/etc/passwd`, `/etc/shadow` o `/etc/sudoers`, te conviertes en root editando una línea. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(writable-sensitive-files\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.